

1. Ogólne założenia systemu

Aplikacja Nawigacja to program na platformę Windows 10/11 stworzony w technologii Universal Windows Platform (UWP) z wykorzystaniem języka C# i interfejsu XAML. Jej celem jest umożliwienie użytkownikowi wyszukania miejsca docelowego, wyznaczenia do niego trasy drogowej oraz wyświetlenia odległości w linii prostej od bieżącej pozycji GPS.

Aplikacja korzysta z usług Bing Maps poprzez wbudowane API systemu Windows: Windows.UI.Xaml.Controls.Maps do wyświetlania mapy oraz Windows.Services.Maps do geolokalizacji i wyznaczania tras.

2. Schemat działania

Aplikacja podzielona jest na dwie strony (Page) oraz jedną klasę pomocniczą. Każda strona odpowiada za oddzielną część działania aplikacji:

- 1) MainPage – strona główna aplikacji, zawiera mapę i pasek poleceń. Wyświetla trasę, znaczniki i oblicza odległość po powrocie ze strony Koordynaty
- 2) Koordynaty – strona wyszukiwania. Pobiera bieżącą pozycję GPS użytkownika oraz umożliwia wyszukanie adresu docelowego przez geokodowanie
- 3) DaneGeograficzne – statyczna klasa pomocnicza przechowująca dane o punkcie startowym i docelowym, dostępna z obu stron bez przekazywania parametrów

Plik	Odpowiedzialność
App.xaml / App.xaml.cs	Klasa startowa – inicjalizacja aplikacji i nawigacja do MainPage
MainPage.xaml	Definicja interfejsu strony głównej: mapa, CommandBar z przyciskami
MainPage.xaml.cs	Logika strony głównej: rysowanie trasy, znaczników, obliczanie odległości
Koordynaty.xaml	Definicja interfejsu strony wyszukiwania: pola tekstowe, przyciski
Koordynaty.xaml.cs	Logika wyszukiwania: pobieranie GPS, geokodowanie adresu
DaneGeograficzne.cs	Klasa statyczna – współdzielone dane między stronami

Nawigacja między stronami odbywa się przy użyciu wbudowanego mechanizmu UWP (Frame.Navigate / Frame.GoBack):

```
MainPage --> [przycisk 'koordynaty'] --> Koordynaty
Koordynaty --> [przycisk 'Mapa'] --> MainPage (Frame.GoBack)
```

Po powrocie do MainPage wywoływana jest metoda OnNavigatedTo, która sprawdza czy oba punkty zostały ustawione i jeśli tak, rysuje na mapie linię prostą, znaczniki oraz wyznacza trasę drogową.

3. Klasa DaneGeograficzne

Klasa DaneGeograficzne pełni rolę wspólnego magazynu danych dla obu stron aplikacji. Dzięki temu, że wszystkie właściwości są statyczne, strona MainPage może odczytać dane wprowadzone na stronie Koordynaty bez konieczności przekazywania ich przez parametry nawigacji.

```
internal class DaneGeograficzne
{
    public static BasicGeoposition pktStartowy { get; set; }
    public static BasicGeoposition pktDocelowy { get; set; }
    public static string opisCelu { get; set; } = null;
}
```

Na początku działania aplikacji właściwości pktStartowy i pktDocelowy mają wartości domyślne (0, 0). MainPage sprawdza to przed próbą rysowania trasy, jeżeli obie wartości są różne od zera, oznacza to że użytkownik uzupełnił dane na stronie Koordynaty.

Właściwość	Opis
pktStartowy	Struktura BasicGeoposition – bieżąca pozycja GPS użytkownika, zapisywana na stronie Koordynaty
pktDocelowy	Struktura BasicGeoposition – współrzędne wyszukanego adresu docelowego
opisCelu	Tekst wpisany przez użytkownika jako adres celu, przechowywany do ewentualnego wyświetlenia

4. Strona główna (MainPage)

Strona główna zawiera kontrolkę MapControl zajmującą większość ekranu oraz CommandBar z przyciskami akcji u dołu. Logika strony obsługuje rysowanie trasy, znaczników, przełączanie trybu mapy oraz nawigację do strony Koordynaty.

Inicjalizacja – konstruktor i token API

W konstruktorze strony ustawiany jest token dostępowy do usług Bing Maps, który jest wymagany do działania mapy, geokodowania i wyznaczania tras:

```
public MainPage()
{
    this.InitializeComponent();
    MapService.ServiceToken = "qEhmCU
```

OnNavigatedTo – rysowanie trasy i znaczników

Metoda OnNavigatedTo wywoływana jest za każdym razem gdy użytkownik wraca na stronę MainPage. Na początku sprawdzamy czy oba punkty geograficzne zostały ustawione. Jeśli tak,

tworzymy przerywaną linię prostą między nimi (linia lotu ptaka), dodajemy dwa znaczniki (pinezki) na mapie i centrujemy widok na punkcie startowym. Następnie wywołujemy metodę obliczającą odległość oraz metodę wyznaczającą trasę drogową.

```
protected override async void OnNavigatedTo(NavigationEventArgs e)
{
    base.OnNavigatedTo(e);

    if (DaneGeograficzne.pktStartowy.Longitude != default(double) && DaneGeograficzne.pktStartowy.Latitude != default(double) &&
        DaneGeograficzne.pktDocelowy.Longitude != default(double) && DaneGeograficzne.pktDocelowy.Latitude != default(double))
    {
        Geopoint startowyGeopoint = new Geopoint(DaneGeograficzne.pktStartowy);

        Geopoint docelowyGeopoint = new Geopoint(DaneGeograficzne.pktDocelowy);

        MapPolyline trasaLotem = new MapPolyline
        {
            StrokeColor = Windows.UI.Colors.Black,
            StrokeThickness = 3,
            StrokeDashed = true,
            Path = new Geopath(new List<BasicGeoposition> {
                DaneGeograficzne.pktStartowy,
                DaneGeograficzne.pktDocelowy
            })
        };

        mojaMapa.MapElements.Add(trasaLotem);
        MapIcon znacznikStart = new MapIcon
        {
            Location = startowyGeopoint,
            Title = "Tu jestem!"
        };

        MapIcon znacznikDocelowy = new MapIcon
        {
            Location = docelowyGeopoint,
            Title = "Cel podróży"
        };

        mojaMapa.MapElements.Add(znacznikStart);
        mojaMapa.MapElements.Add(znacznikDocelowy);

        await mojaMapa.TrySetViewAsync(startowyGeopoint, 8);
        await ObliczOdleglosc();
    }
    Trasa();
}
```

Na początku sprawdzamy czy właściwości klasy DaneGeograficzne mają wartości różne od domyślnych (0). Jeżeli tak, tworzymy obiekty Geopoint na podstawie zapisanych współrzędnych. Następnie tworzymy obiekt MapPolyline – przerywaną linię rysowaną na mapie. Dodajemy dwa obiekty MapIcon jako znaczniki (pinezki). Na końcu centrujemy widok mapy na punkcie startowym z poziomem przybliżenia 8 i wywołujemy metody obliczające odległość oraz wyznaczające trasę drogową.

ObliczOdleglosc – wzór Haversine

Metoda oblicza odległość w linii prostej między bieżącą pozycją GPS a punktem docelowym, używając wzoru Haversine uwzględniającego kulistość Ziemi. Wynik wyświetlany jest użytkownikowi w oknie dialogowym `MessageDialog`.

```
private double ObliczOdleglosc(BasicGeoposition punktA, BasicGeoposition punktB)
{
    const double earthRadius = 6371;

    var lat1 = ToRadians(punktA.Latitude);
    var lon1 = ToRadians(punktA.Longitude);
    var lat2 = ToRadians(punktB.Latitude);
    var lon2 = ToRadians(punktB.Longitude);

    var dlon = lon2 - lon1;
    var dlat = lat2 - lat1;

    var a = Math.Pow(Math.Sin(dlat / 2), 2) + Math.Cos(lat1) * Math.Cos(lat2) * Math.Pow(Math.Sin(dlon / 2), 2);
    var c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));

    return earthRadius * c * 1000;
}
```

Metoda przyjmuje dwa punkty geograficzne. Zamienia stopnie na radiany metodą pomocniczą `ToRadians`. Oblicza różnicę długości i szerokości geograficznych. Następnie stosuje wzór Haversine: oblicza zmienną 'a' jako sumę kwadratów sinusów połowy różnic, a zmienną 'c' jako podwojoną funkcję `atan2`. Mnożąc przez promień Ziemi (6371 km) i przez 1000 otrzymujemy wynik w metrach, który dzielony przez 1000 daje kilometry wyświetlane w oknie dialogowym.

Trasa – wyznaczanie trasy drogowej

Metoda `Trasa` wysyła zapytanie do serwisu Bing Maps o wyznaczenie trasy samochodowej między punktem startowym a docelowym. W przypadku sukcesu trasa jest dodawana do mapy i widok jest dopasowywany do jej granic. W przypadku błędu wyświetlany jest komunikat.

```
private async void Trasa()
{
    if (DaneGeograficzne.pktStartowy.Longitude != default(double) && DaneGeograficzne.pktStartowy.Latitude != default(double) &&
        DaneGeograficzne.pktDocelowy.Longitude != default(double) && DaneGeograficzne.pktDocelowy.Latitude != default(double))
    {
        BasicGeoposition startowyPos = DaneGeograficzne.pktStartowy;
        BasicGeoposition docelowyPos = DaneGeograficzne.pktDocelowy;

        Geopoint startowyGeopoint = new Geopoint(startowyPos);
        Geopoint docelowyGeopoint = new Geopoint(docelowyPos);

        MapRouteFinderResult wynik = await MapRouteFinder.GetDrivingRouteAsync(startowyGeopoint, docelowyGeopoint);

        if (wynik.Status == MapRouteFinderStatus.Success)
        {
            MapRoute trasa = wynik.Route;

            MapRouteView trasaWidok = new MapRouteView(trasa);

            mojaMapa.Routes.Add(trasaWidok);

            await mojaMapa.TrySetViewBoundsAsync(trasa.BoundingBox, null, Windows.UI.Xaml.Controls.Maps.MapAnimationKind.None);
        }
        else
        {
            MessageDialog dialog = new MessageDialog("Nie udało się wyznaczyć trasy.");
            await dialog.ShowAsync();
        }
    }
}
```

Najpierw sprawdzamy warunek analogiczny jak w OnNavigatedTo – oba punkty muszą być ustawione. Wywołujemy MapRouteFinder.GetDrivingRouteAsync, który asynchronicznie kontaktuje się z Bing Maps. Jeżeli status odpowiedzi to Success, tworzymy MapRouteView z otrzymanej trasy i dodajemy ją do kolekcji Routes mapy. Następnie dopasowujemy widok do obszaru trasy metodą TrySetViewBoundsAsync. Jeżeli wyznaczenie trasy się nie powiedzie, wyświetlamy okno dialogowe z komunikatem o błędzie.

powMapa / pomMapa – zoom mapy

Dwie proste metody obsługujące przyciski powiększania i pomniejszania mapy. Każda sprawdza czy nie zostanie przekroczony minimalny lub maksymalny poziom przybliżenia przed zmianą wartości ZoomLevel:

```
private void powMapa(object sender, RoutedEventArgs e)
{
    if (mojaMapa.ZoomLevel < mojaMapa.MaxZoomLevel)
    {
        mojaMapa.ZoomLevel++;
    }
}

private void pomMapa(object sender, RoutedEventArgs e)
{
    if (mojaMapa.ZoomLevel > mojaMapa.MinZoomLevel)
    {
        mojaMapa.ZoomLevel--;
    }
}
```

trybMapy – przełączanie widoku mapy

Metoda przełącza styl mapy między widokiem drogowym (Road) a satelitarnym z nakładką dróg (AerialWithRoads). Jednocześnie aktualizuje etykietę i ikonę przycisku AppBarButton, który wywołał akcję:

```
private void trybMapy(object sender, RoutedEventArgs e)
{
    AppBarButton ab = sender as AppBarButton;
    if (ab != null)
    {
        FontIcon fIcon = new FontIcon();
        fIcon.FontFamily = new FontFamily("Segoe UI");

        if (mojaMapa.Style == MapStyle.Road)
        {
            mojaMapa.Style = MapStyle.AerialWithRoads;
            ab.Label = "mapa";
            fIcon.Glyph = "M";
        }
        else
        {
            mojaMapa.Style = MapStyle.Road;
            ab.Label = "satelita";
            fIcon.Glyph = "S";
        }

        ab.Icon = fIcon;
    }
}
```

Na początku rzutujemy obiekt sender na AppBarButton, żeby uzyskać dostęp do jego właściwości. Sprawdzamy aktualny styl mapy. Jeżeli jest drogowy (Road), przełączamy na AerialWithRoads i zmieniamy etykietę przycisku na 'mapa' oraz glif na literę M. W

przeciwnym wypadku przywracamy styl drogowy i zmieniamy etykietę z powrotem na 'satelita'. Na końcu przypisujemy nową ikonę do przycisku.

5. Strona koordynat (Koordynaty)

Strona Koordynaty umożliwia dwie rzeczy: pobranie bieżącej pozycji GPS urządzenia oraz wyszukanie adresu docelowego i zamienienie go na współrzędne geograficzne (geokodowanie). Dane obu punktów zapisywane są do klasy DaneGeograficzne, skąd pobiera je strona MainPage.

GdzieJaNaMapie – pobieranie pozycji GPS

Metoda wywoływana jest automatycznie w konstruktorze strony, zaraz po załadowaniu widoku. Tworzy obiekt Geolocator z żadaną dokładnością 1 metra i asynchronicznie pobiera bieżącą pozycję urządzenia. Pobrane współrzędne wyświetla w polu tekstowym tbGPS oraz zapisuje do DaneGeograficzne.pktStartowy:

```
private async void GdzieJaNaMapie()
{
    Geolocator mójGPS = new Geolocator { DesiredAccuracyInMeters = 1 };

    Geoposition mojeZGPS = await mójGPS.GetGeopositionAsync();
    tbGPS.Text = $"Moje położenie: {mojeZGPS.Coordinate.Point.Position.Latitude}, {mojeZGPS.Coordinate.Point.Position.Longitude}";

    DaneGeograficzne.pktStartowy = new BasicGeoposition
    {
        Latitude = mojeZGPS.Coordinate.Point.Position.Latitude,
        Longitude = mojeZGPS.Coordinate.Point.Position.Longitude
    };
}
```

Tworzymy obiekt Geolocator z właściwością DesiredAccuracyInMeters ustawioną na 1, co oznacza żądanie jak najdokładniejszego odczytu GPS. Metoda GetGeopositionAsync asynchronicznie czeka na odpowiedź modułu lokalizacji. Po otrzymaniu danych wyświetlamy szerokość i długość geograficzną w polu tekstowym oraz zapisujemy je do statycznej właściwości pktStartowy klasy DaneGeograficzne.

Szukaj_Click – geokodowanie adresu

Metoda obsługuje kliknięcie przycisku Szukaj. Pobiera wpisany adres z pola tekstowego i wysyła zapytanie do serwisu Bing Maps o znalezienie pasującej lokalizacji. Jako wskazówkę lokalizacyjną przekazuje punkt startowy GPS, co zwiększa trafność wyników wyszukiwania. W przypadku sukcesu współrzędne pierwszego wynik zapisywane są do DaneGeograficzne i wyświetlane na ekranie.


```

private async void Szukaj_Click(object sender, RoutedEventArgs e)
{
    string szukanyAdres = txAdres.Text;

    if (string.IsNullOrEmpty(szukanyAdres))
    {
        return;
    }

    MapLocationFinderResult wynik = await MapLocationFinder.FindLocationsAsync(szukanyAdres, new Geopoint(DaneGeograficzne.pktStartowy), 3);

    if (wynik.Status == MapLocationFinderStatus.Success && wynik.Locations.Count > 0)
    {
        DaneGeograficzne.pktDocelowy = wynik.Locations[0].Point.Position;
        DaneGeograficzne.opisCelu = szukanyAdres;

        tbDlg.Text = $"Długość geograficzna: {DaneGeograficzne.pktDocelowy.Longitude}";
        tbSzer.Text = $"Szerokość geograficzna: {DaneGeograficzne.pktDocelowy.Latitude}";
    }
    else
    {
        tbDlg.Text = "Nie można znaleźć lokalizacji.";
        tbSzer.Text = "";
    }
}
}

```

Na początku pobieramy tekst z pola txAdres. Jeżeli pole jest puste, metoda kończy działanie bez żadnej akcji. Następnie wywołujemy MapLocationFinder.FindLocationsAsync przekazując wpisany adres, bieżący punkt GPS jako odpowiedź lokalizacyjną oraz limit 3 wyników. Sprawdzamy status odpowiedzi – jeśli jest Success i lista wyników niepusta, bierzemy pierwszy element (wynik.Locations[0]) i zapisujemy jego pozycję do DaneGeograficzne.pktDocelowy. Długość i szerokość geograficzną wyświetlamy w odpowiednich polach tekstowych. W przypadku błędu lub braku wyników wyświetlamy komunikat w polu tbDlg.

Powrot_Click – powrót do mapy

Obsługuje kliknięcie przycisku Mapa. Sprawdza czy stos nawigacji zawiera poprzednią stronę i jeśli tak, wykonuje cofnięcie:

```

private void Powrot_Click(object sender, RoutedEventArgs e)
{
    if (Frame.CanGoBack)
    {
        Frame.GoBack();
    }
}

```